could u tell me will anyone of these will 90% win the build with ai hackathon from devpost.... if u have suggestion u can add...

No project can honestly be guaranteed a **90% chance of winning**—hackathons depend on the other submissions, judging, polish, and demo quality. But based on the official Devpost criteria—**Design, Potential Impact, Innovation/Idea, and Presentation**, equally weighted—the strongest choice from your four ideas is **Cognitive Debt Auditor**, with **EdgeCase** as the best alternative.[1] [2]

My ranking

Rank	Idea	Winning potential	Main reason
1	**Cognitive Debt Auditor**	Highest	Directly matches the hackathon's AI-best-practices theme and has a memorable problem.
2	**EdgeCase**	Very high	Easy to understand and visually impressive during a live demo.
3	**SpecPilot**	High	Strong alignment with the Flipped Interaction Method, but many AI planning tools may look similar.
4	**Doc-to-Skill Compiler**	Good	Useful and technically credible, but documentation-to-prompt tools are becoming common.

The official rules emphasize that the project must be a coherent product—not merely a technical proof of concept—and must make a credible case for solving a real problem for a specific audience.[1]

Best choice: Cognitive Debt Auditor

I would choose **Cognitive Debt Auditor**, but I would narrow it significantly.

Stronger version

> **Before an AI-generated code change is merged, Cognitive Debt Auditor checks whether the developer understands it and detects hidden risks.**

The tool should do only one complete workflow:

1. User uploads or selects a Git diff.
2. AI identifies the most important changed functions.
3. AI asks the developer three targeted questions.

4. The developer answers by text or voice.
5. The tool scores understanding and highlights risky areas.
6. It generates:
    - `COGNITIVE_MAP.md`
    - a risk report
    - suggested tests
    - a final “safe to review” or “needs deeper review” result.

This is stronger than simply generating a code explanation because it demonstrates **human–AI accountability**. The developer must prove understanding rather than passively reading an AI summary.

Why judges may remember it

- It solves a problem created by AI coding tools.
- It is directly relevant to developers.
- It is more original than a generic chatbot.
- It can be demonstrated clearly in two minutes.
- It connects AI productivity with software quality and responsibility.

Do not claim that the tool proves code is safe. Say that it **helps reveal areas the developer may not understand**.

Best demo alternative: EdgeCase

Choose **EdgeCase** if you want the safest and most visually impressive demo.

A good demo would show:

1. An apparently correct function.
2. The AI creates adversarial inputs.
3. Several tests fail.
4. The tool explains the failures.
5. It generates a patch.
6. The tests pass after the patch.

This is easy for judges to understand immediately. The weakness is that “AI code tester” is a crowded concept, so you need a unique angle.

Make EdgeCase more original

Focus on **AI-generated code specifically**, not arbitrary code:

> "EdgeCase is a pre-merge safety check for code written by AI agents."

Add a clear "AI blind spot" report covering:

- Missing input validation.
- Network failures.
- Race conditions.
- Unicode and localization issues.
- Permission mistakes.
- Data leakage.
- Incorrect retry behavior.

Ideas I would avoid as the main project

Doc-to-Skill Compiler

It is useful, but the demo could look like:

> "Give us documentation, and we generate a better prompt file."

That may be judged as prompt engineering rather than a complete product. It also depends heavily on scraping and parsing documentation reliably.

SpecPilot

This fits the Flipped Interaction Method very well, but many competitors may build AI idea-to-PRD applications. To stand out, it would need to generate something executable—not only Markdown documents.

A stronger version would:

- Ask clarifying questions.
- Generate `scope.md`, `prd.md`, and `spec.md`.
- Create a runnable starter repository.
- Run automated checks against the requirements.
- Show which requirements are implemented and which are missing.

My recommendation

Build a hybrid, but keep the product focused:

UnderstandGuard

Tagline: “AI can write the code. UnderstandGuard makes sure you understand the change.”

Core workflow:

- Git diff input.
- AI-generated reverse interview.
- Developer answers.
- Risk and understanding score.
- Edge-case test suggestions.
- Cognitive map export.

This combines the strongest parts of **Cognitive Debt Auditor** and **EdgeCase** without becoming too large. The main feature should remain the reverse interview; edge-case testing should support it rather than become a second unrelated product.

Suggested two-minute video

- **0:00–0:15:** Show an AI-generated pull request and explain the problem: “The code works, but the developer may not understand it.”
- **0:15–0:40:** Upload the Git diff.
- **0:40–1:10:** The AI asks three difficult, code-specific questions.
- **1:10–1:25:** The developer gives an incomplete answer.
- **1:25–1:40:** The tool identifies the risky function and generates an edge-case test.
- **1:40–1:55:** Show the cognitive map and review report.
- **1:55–2:00:** End with: “UnderstandGuard turns AI-generated code from something you accept into something you can explain.”

How to maximize your chance

- Build one polished workflow instead of many features.
- Use a real-looking Git diff with a subtle bug.
- Make every AI question reference an actual line or function.
- Show deterministic fallback data if the API fails during the demo.
- Include `scope.md`, `prd.md`, and `spec.md` in the repository.
- Add a short architecture diagram.
- Show the product working end-to-end in the video, because presentation explicitly evaluates clarity and working demonstration.[3] [1]

- Avoid calling it “100% accurate,” “zero hallucinations,” or “guaranteed secure.”
- Make the first 15 seconds understandable to a non-expert judge.

My final recommendation: **Build Cognitive Debt Auditor as UnderstandGuard, with one small EdgeCase feature included.** That gives you the best balance of originality, theme alignment, practical impact, and demo clarity.

⁂

1. https://learn-ai-basics.devpost.com/rules
2. hackideas.md
3. https://learn-ai-basics.devpost.com/
4. https://learn-ai-basics.devpost.com/?ref_content=featured&ref_feature=challenge&ref_medium=portfolio
5. https://help.devpost.team/article/230-how-to-set-up-judging
6. https://aibuildathon.devpost.com/
7. https://build-with-ai.devpost.com/
8. https://info.devpost.com/blog/understanding-hackathon-submission-and-judging-criteria
9. https://info.devpost.com/blog/hackathon-judging-tips
10. https://developerweek-2026-hackathon.devpost.com/
11. https://help.devpost.com/article/103-how-to-judge-an-online-hackathon
12. https://help.devpost.com/article/191-judging-at-devpost-hackathons
13. https://help.devpost.com/article/131-setting-up-judging
14. https://help.devpost.com/article/64-judging-public-voting
15. https://learn-ai-basics.devpost.com/details/dates
16. https://learn-ai-basics.devpost.com/resources